Tarea 03: Implementación de Regresión Logística con PyTorch para Clasificación de Calidad del Vino

Jose Miguel Gonzalez Barrantes Emmanuel Rodríguez Rivas
Carné: 2023087564 Carné: 2023146102
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Profesor: Steven Andrey Pacheco Portugués
Curso: IC6200 Inteligencia Artificial
Fecha: Jueves 9 de abril de 2026

***Resumen*—En este trabajo se implementó un modelo de regresión logística utilizando PyTorch para clasificar la calidad del vino tinto como buena o mala a partir del dataset *Wine Quality Red* de Kaggle. Primero se realizó un análisis exploratorio de datos para estudiar la estructura del conjunto, revisar tipos de variables, valores nulos, duplicados, distribuciones, valores atípicos y correlaciones. Luego, la variable objetivo original fue transformada a una representación binaria, considerando como vinos buenos aquellos con calidad mayor o igual a 6. Después se seleccionaron las seis variables con mayor correlación absoluta respecto a la clase objetivo, se dividieron los datos en entrenamiento, validación y prueba mediante muestreo estratificado, y se aplicó tratamiento de outliers con IQR junto con estandarización. Finalmente, se definió y entrenó un modelo de regresión logística en PyTorch y se ejecutaron diez experimentos variando la tasa de aprendizaje, el tamaño de batch y la cantidad de épocas. Los resultados mostraron que el mejor experimento según F1 sobre validación fue el experimento 2 (F1 de 0.7440). Al evaluar este modelo final con el conjunto de prueba, se obtuvo un F1 de 0.7596 y una exactitud de 0.7549, demostrando una excelente capacidad de generalización sin sobreajuste.**

***Index Terms*—Regresión logística, PyTorch, clasificación binaria, Wine Quality Red, análisis exploratorio de datos, preprocesamiento, matriz de confusión**

I. Introducción

La regresión logística es uno de los modelos más utilizados dentro del aprendizaje supervisado cuando el objetivo consiste en clasificar observaciones en dos categorías, como se se tratara de 0s y 1s binarios. A diferencia de la regresión lineal, este modelo no busca predecir un valor continuo, sino estimar la probabilidad de pertenecer a una clase específica. Por esto, es especialmente útil para problemas de clasificación binaria como detección de fraude, diagnóstico médico o, como en este caso, clasificación de calidad del vino.

En esta tarea se trabajó con el dataset *Wine Quality Red*, tomado de Kaggle, con el propósito de determinar si un vino tinto puede ser considerado bueno o malo a partir de sus propiedades fisicoquímicas. Para desarrollar el flujo completo, primero se realizó un análisis exploratorio del conjunto de datos, después se aplicaron distintas etapas de preprocesamiento y finalmente se implementó el modelo con PyTorch, además de un análisis de todo lo que se realizó.

El objetivo principal no fue solamente entrenar un clasificador, sino también comprender cada una de las etapas previas necesarias para construirlo correctamente, tal y como se vió en clase. Por esa razón, se revisaron aspectos como la presencia de duplicados, la distribución de las variables, la existencia de outliers, la relación entre características y target, la selección de variables relevantes, la división estratificada del dataset y el escalado de las entradas. Todo esto mencionado es fundamental para que el modelo trabaje con datos limpios y así resulte en un resultado de calidad.

II. Descripción del dataset

El conjunto de datos utilizado fue *Wine Quality Red*, el cual contiene información fisicoquímica de vinos tintos y una variable de calidad asociada. Originalmente el dataset presentó 1599 observaciones y 12 columnas. Las variables de entrada correspondieron a medidas numéricas como *fixed acidity*, *volatile acidity*, *citric acid*, *residual sugar*, *chlorides*, *free sulfur dioxide*, *total sulfur dioxide*, *density*, *pH*, *sulphates* y *alcohol*. La variable *quality* representó la calidad del vino en una escala discreta.

Como la tarea estaba planteada como un problema de clasificación binaria, la variable *quality* fue transformada tiempo después, en una nueva variable llamada `quality_binary`, donde se asignó el valor 1 a los vinos con calidad mayor o igual a 6, y 0 en caso contrario, como si se tratara de un resultado binario.

III. Lectura y exploración inicial del dataset

III-A. Carga del dataset

Para comenzar el análisis, el archivo CSV fue cargado en un *DataFrame* de Pandas. Además, se revisaron las primeras filas y el tamaño general del conjunto.

```
import pandas as pd
import numpy as np

df = pd.read_csv("winequality-red.csv")

print("Primeras 5 filas del dataset:")
display(df.head())

print("Cantidad de observaciones (filas):", df.shape
    [0])
print("Cantidad de variables (columnas):", df.shape
    [1])
```

Listing 1: Carga inicial del dataset

Al cargar el dataset se observó que contenía 1599 filas y 12 columnas. Las primeras observaciones mostraron una estructura completamente numérica, lo cual facilitó el análisis exploratorio inicial y el posterior modelado.

III-B. *Información general del dataset*

Después de la carga inicial, se revisó la estructura del conjunto con `df.info()` para verificar los tipos de dato y la presencia de valores no nulos.

```
df.info()
```

Listing 2: Información general del dataset

Se encontró que las 11 variables predictoras eran de tipo `float64`, mientras que la variable `quality` era de tipo `int64`. Además, todas las columnas presentaban 1599 valores no nulos, lo cual indicó desde el inicio que no existían datos faltantes. Es decir cada observación tenía un valor asignado para todas las columnas.

III-C. *Estadísticas descriptivas*

A continuación se calcularon estadísticas descriptivas para tener una idea más clara del rango, dispersión y tendencia central de cada variable numérica.

```
df.describe()
```

Listing 3: Descripción estadística básica del dataset

A partir de esta descripción se observó, por ejemplo, que la variable *alcohol* tenía una media aproximada de 10.42 y un rango entre 8.4 y 14.9, mientras que *quality* presentaba una media cercana a 5.64. También se notó que algunas variables como *residual sugar*, *total sulfur dioxide* y *chlorides* tenían máximos bastante alejados de sus cuartiles superiores, lo que ya daba indicios de posibles valores atípicos.

III-D. *Valores nulos*

Como parte de la revisión previa, se verificó si existían valores nulos en el conjunto de datos.

```
print("Valores nulos por columna:")
print(df.isnull().sum())

print("Cantidad total de valores nulos en el dataset
    :",
      df.isnull().sum().sum())
```

Listing 4: Verificación de valores nulos

No se encontraron valores nulos en ninguna columna. Esto fue importante porque permitió continuar con el flujo sin necesidad de aplicar técnicas de imputación o eliminación de registros incompletos.

III-E. *Valores duplicados*

También se revisó si el dataset contenía filas repetidas, ya que esto puede introducir sesgos y hacer que el modelo parezca funcionar mejor de lo que realmente lo hace, es decir un favoritismo a algún elemento.

```
print("Cantidad de filas duplicadas:", df.duplicated
    ().sum())
```

Listing 5: Detección de filas duplicadas

Se detectaron 240 filas duplicadas. Debido a que esta cantidad era considerable, se decidió eliminarlas antes de cualquier división entre entrenamiento, validación y prueba.

```
df = df.drop_duplicates()
print("Nuevo tamaño del dataset:", df.shape)
```

Listing 6: Eliminación de filas duplicadas

Luego de esta limpieza, el tamaño del dataset pasó a ser de 1359 observaciones y 12 columnas.

IV. Análisis de la variable objetivo

IV-A. *Transformación binaria de la calidad*

Dado que el objetivo del trabajo fue clasificar los vinos como buenos o malos, la variable original `quality` se convirtió en una versión binaria.

```
df["quality_binary"] = (df["quality"] >= 6).astype(
    int)

print("\nDistribución de la variable binaria:")
print(df["quality_binary"].value_counts())

target = "quality_binary"

features = [col for col in df.columns
            if col not in ["quality_binary", "
                quality"]]

X = df[features]
y = df[target]
```

Listing 7: Creación de la variable objetivo binaria

Con este criterio, 719 vinos fueron clasificados como buenos y 640 como malos. La distribución resultante fue relativamente cercana, por lo que no se observaron problemas severos de desbalance de clases. Estos dos valores obtenidos dan a entender que está correctamente balanceado el ajuste realizado.

V. Análisis exploratorio de datos

V-A. *Histogramas*

Los histogramas permitieron estudiar la forma general de las distribuciones para todas las variables numéricas del dataset.

```
import matplotlib.pyplot as plt

df.drop(columns=["quality_binary"]).hist(
    figsize=(14, 10), bins=20, edgecolor="black"
)
plt.suptitle("Histogramas de las variables numéricas
    ", y=1.02)
plt.tight_layout()
plt.show()
```

Listing 8: Generación de histogramas

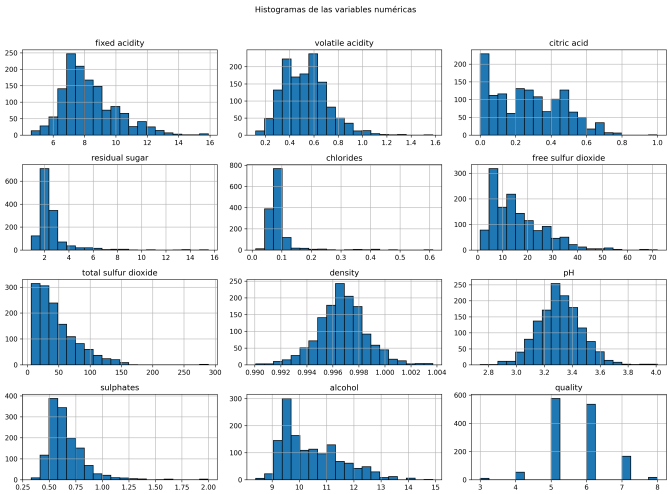

Figura 1: Histogramas de las variables numéricas del dataset.

A partir de los histogramas se observó que las variables *density* y *pH* presentaban un comportamiento más cercano a una distribución de campana. En cambio, *quality* no mostraba una forma normal, lo cual es lógico porque se trata de una variable discreta. Varias de las demás variables presentaron sesgo hacia la derecha, especialmente aquellas donde luego también se identificaron outliers en los boxplots.

V-B. *Diagramas de caja*

Los diagramas de caja fueron utilizados para identificar dispersión y valores atípicos en cada una de las variables.

```
import matplotlib.pyplot as plt
import seaborn as sns

columnas = df.drop(columns=["quality_binary"]).
    columns

fig, axes = plt.subplots(nrows=4, ncols=3, figsize
    =(16, 14))
axes = axes.flatten()

for i, col in enumerate(columnas):
    sns.boxplot(x=df[col], ax=axes[i])
    axes[i].set_title(f"{col}")
    axes[i].set_xlabel("")

for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle("Boxplots de las variables numéricas",
    fontsize=16)
plt.tight_layout()
plt.show()
```

Listing 9: Generación de boxplots

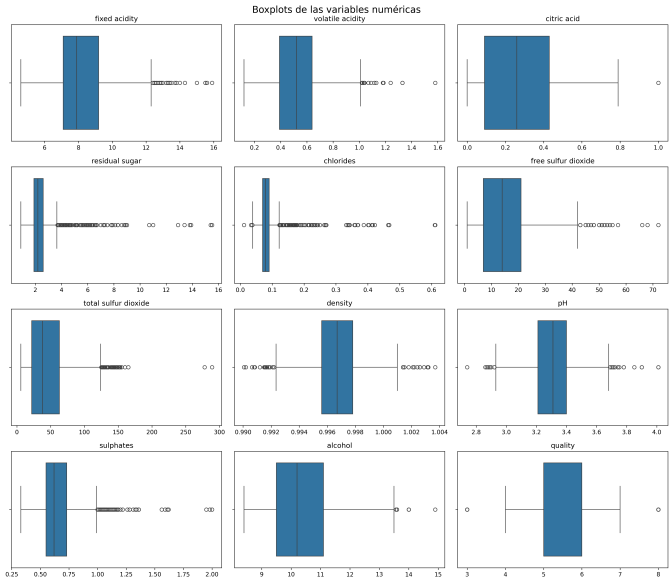

Figura 2: Boxplots de las variables numéricas del dataset.

En los boxplots se encontró que variables como *residual sugar*, *chlorides*, *free sulfur dioxide*, *total sulfur dioxide* y *sulphates* presentaban una cantidad importante de valores atípicos hacia valores altos. Por otro lado, *density* y *pH* mostraron distribuciones más compactas, aunque no completamente libres de extremos.

V-C. *Diagramas de dispersión respecto a la variable objetivo*

Después se construyeron diagramas de dispersión para visualizar cómo se relacionaba cada característica con la variable objetivo binaria.

```
import matplotlib.pyplot as plt
import seaborn as sns
import math

target = "quality_binary"
features = [col for col in df.columns if col !=
    target]

n = len(features)
cols = 3
rows = math.ceil(n / cols)

plt.figure(figsize=(15, 5 * rows))

for i, col in enumerate(features, 1):
    plt.subplot(rows, cols, i)
    sns.scatterplot(data=df, x=col, y=target, hue=
        target, alpha=0.6)
    plt.title(col)

plt.tight_layout()
plt.show()
```

Listing 10: Diagramas de dispersión respecto al target

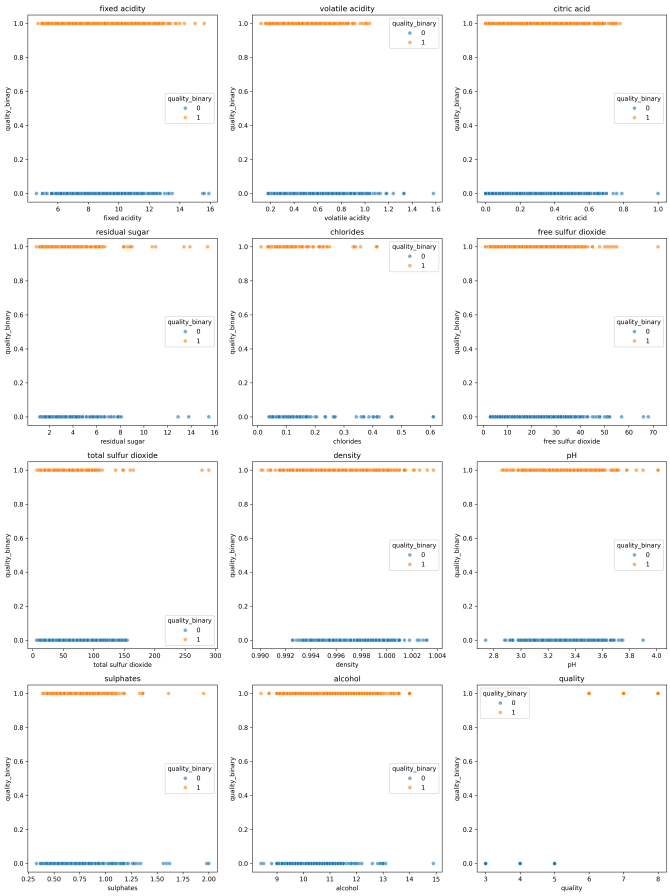

Figura 3: Diagramas de dispersión de las variables respecto a `quality_binary`.

En esta visualización se apreció que variables como *alcohol* y *volatile acidity* mostraban una separación más visible entre las clases. Eso sugirió que podían ser especialmente útiles para la predicción. Además, se confirmó nuevamente la presencia de valores extremos en variables como *residual sugar* y *chlorides*. Debido a ello se decidió aplicar posteriormente un tratamiento de outliers mediante IQR.

V-D. Matriz de correlación

Para complementar el análisis, se calculó una matriz de correlación y se representó mediante un mapa de calor.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 8))
corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True, cmap="coolwarm",
            fmt=".2f", linewidths=0.5)
plt.title("Matriz de correlación")
plt.tight_layout()
plt.show()
```

Listing 11: Matriz de correlación con mapa de calor

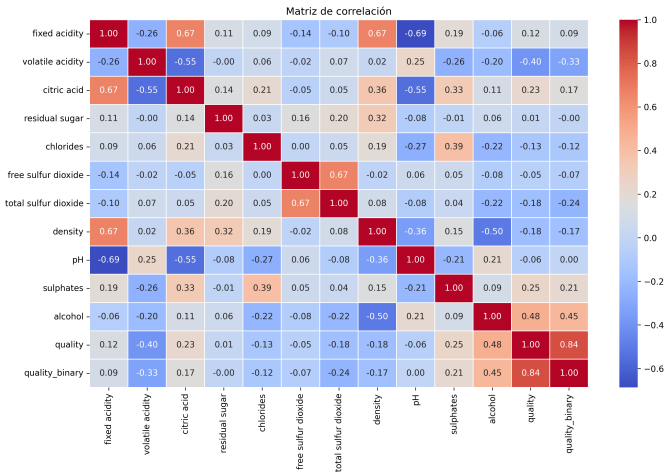

Figura 4: Mapa de calor de la matriz de correlación.

La matriz mostró que *alcohol* era la variable con mayor correlación positiva respecto a `quality_binary`, con un valor de 0.45. En cambio, *volatile acidity* presentó una correlación negativa moderada de aproximadamente -0.33. También destacaron *total sulfur dioxide* con -0.24 y *sulphates* con 0.21.

Además, se observaron correlaciones importantes entre variables predictoras, por ejemplo entre *fixed acidity* y *citric acid* (0.67), *fixed acidity* y *density* (0.67), *free sulfur dioxide* y *total sulfur dioxide* (0.67), y una correlación negativa fuerte entre *fixed acidity* y *pH* (-0.69). Esto sugirió que algunas variables podían aportar información redundante.

V-E. Selección de las seis variables principales

Para la selección de características se utilizó como criterio principal el coeficiente de correlación de Pearson entre cada variable numérica y la variable objetivo que se eligió anteriormente.

```
corr_target = df.drop(columns=["quality"]).corr(
    numeric_only=True)["quality_binary"] \
    .drop("quality_binary") \
    .sort_values(key=lambda s: s.abs(), ascending=
        False)

print(corr_target)
```

Listing 12: Búsqueda de variables con mayor correlación

Se seleccionaron las seis variables con mayor valor absoluto de correlación: *alcohol* (0.435), *volatile acidity* (-0.321), *total sulfur dioxide* (-0.232), *sulphates* (0.218), *citric acid* (0.159) y *density* (-0.159). Las demás variables fueron descartadas por presentar una relación más débil con la variable a predecir. Curiosamente esto concuerda con lo que se vio en la matriz de correlación y los distintos diagramas.

```
selected_features = [
    "alcohol",
    "volatile acidity",
    "total sulfur dioxide",
    "sulphates",
    "density",
    "citric acid"
]
```

Listing 13: Selección final de las seis características

Tabla I: Seis variables seleccionadas según correlación absoluta con `quality_binary`

Variable	Correlación
alcohol	0.434751
volatile acidity	-0.321441
total sulfur dioxide	-0.231963
sulphates	0.218072
citric acid	0.159129
density	-0.159110

VI. División y preparación de los datos

VI-A. *División estratificada del dataset*

Una vez definidas las seis características principales, el conjunto fue separado en entrenamiento, validación y prueba utilizando muestreo estratificado. Primero se apartó un 70 % para entrenamiento y un 30 % temporal, el cual luego fue dividido por igual para validación y prueba.

```
from sklearn.model_selection import train_test_split

selected_features = [
    "alcohol",
    "volatile acidity",
    "total sulfur dioxide",
    "sulphates",
    "density",
    "citric acid"
]

X = df[selected_features]
y = df["quality_binary"]

# 70 % entrenamiento, 30 % temporal
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    stratify=y,
    random_state=42
)

# 15 % validación, 15 % prueba
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    stratify=y_temp,
    random_state=42
)
```

Listing 14: División del dataset en entrenamiento, validación y prueba

Las proporciones de clase se mantuvieron prácticamente iguales en los tres subconjuntos, lo que indicó que el proceso de estratificación fue correcto y ayudó a evitar sesgos en la evaluación.

VI-B. *Aplicación del método IQR*

Como el análisis exploratorio mostró presencia de outliers, se decidió aplicar una estrategia de capado usando el rango intercuartílico (IQR). Este procedimiento se calculó únicamente con el conjunto de entrenamiento para evitar fuga de información.

```
X_train_iqr = X_train.copy()
X_val_iqr = X_val.copy()
X_test_iqr = X_test.copy()

for col in selected_features:
    Q1 = X_train[col].quantile(0.25)
    Q3 = X_train[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    X_train_iqr[col] = X_train_iqr[col].clip(lower,
        upper)
    X_val_iqr[col] = X_val_iqr[col].clip(lower,
        upper)
    X_test_iqr[col] = X_test_iqr[col].clip(lower,
        upper)
```

Listing 15: Tratamiento de outliers con IQR

Este enfoque permitió limitar los valores extremos sin necesidad de eliminar filas completas.

VI-C. *Escalado de los datos*

Posteriormente, las variables fueron estandarizadas usando `StandardScaler`. Esta etapa fue importante porque las características tenían rangos distintos, y la regresión logística puede verse afectada si unas variables dominan a otras por escala.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_iqr)
X_val_scaled = scaler.transform(X_val_iqr)
X_test_scaled = scaler.transform(X_test_iqr)
```

Listing 16: Escalado de las variables con StandardScaler

VII. Implementación con PyTorch

VII-A. *Conversión de datos a tensores*

Una vez escaladas las entradas, los subconjuntos se transformaron a tensores de PyTorch para poder utilizarlos dentro del modelo.

```
import torch

X_train_tensor = torch.tensor(X_train_scaled, dtype=
    torch.float32)
X_val_tensor = torch.tensor(X_val_scaled, dtype=
    torch.float32)
X_test_tensor = torch.tensor(X_test_scaled, dtype=
    torch.float32)

y_train_tensor = torch.tensor(y_train.values,
                              dtype=torch.float32).
                                  view(-1, 1)
y_val_tensor = torch.tensor(y_val.values,
                            dtype=torch.float32).
                                view(-1, 1)
y_test_tensor = torch.tensor(y_test.values,
                             dtype=torch.float32).
                                 view(-1, 1)
```

Listing 17: Conversión de datos a tensores

VII-B. *Creación de los Dataset y DataLoader*

Con los tensores ya preparados, se construyeron los objetos `TensorDataset` y `DataLoader` para organizar el entrenamiento por lotes.

```
from torch.utils.data import TensorDataset,
    DataLoader

train_dataset = TensorDataset(X_train_tensor,
    y_train_tensor)
val_dataset = TensorDataset(X_val_tensor,
    y_val_tensor)

train_loader = DataLoader(train_dataset, batch_size
    =32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32,
    shuffle=False)
```

Listing 18: Creación de TensorDataset y DataLoader

VII-C. *Definición del modelo de regresión logística*

El modelo se implementó como una sola capa lineal, adecuada para clasificación binaria.

```
import torch.nn as nn

class LogisticRegressionModel(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.linear = nn.Linear(input_dim, 1)

    def forward(self, x):
        return self.linear(x)
```

Listing 19: Definición del modelo de regresión logística en PyTorch

VIII. Ejecución de los diez experimentos

VIII-A. *Configuración de experimentos*

Para comparar distintas configuraciones, se definieron diez experimentos variando tasa de aprendizaje, tamaño de batch y número de épocas.

```
experiments = [
    {"lr": 0.001, "batch": 16, "epochs": 20},
    {"lr": 0.001, "batch": 32, "epochs": 20},
    {"lr": 0.001, "batch": 64, "epochs": 20},
    {"lr": 0.005, "batch": 16, "epochs": 20},
    {"lr": 0.005, "batch": 32, "epochs": 20},
    {"lr": 0.005, "batch": 64, "epochs": 20},
    {"lr": 0.01,  "batch": 16, "epochs": 20},
    {"lr": 0.01,  "batch": 32, "epochs": 20},
    {"lr": 0.01,  "batch": 64, "epochs": 20},
    {"lr": 0.01,  "batch": 32, "epochs": 50},
]
```

Listing 20: Definición de los diez experimentos

VIII-B. *Loop de entrenamiento y captura de métricas*

En esta sección se procedió con la ejecución de los experimentos. Se integró un diccionario para capturar el historial de la función de pérdida (*Loss*) en cada época, con el fin de graficar posteriormente las curvas de aprendizaje.

```
import torch.optim as optim
from sklearn.metrics import accuracy_score,
    precision_score, recall_score, f1_score,
    roc_auc_score

results = []
loss_history = {}

criterion = nn.BCEWithLogitsLoss()

for i, exp in enumerate(experiments):
    # instanciar el modelo y el optimizador
    model = LogisticRegressionModel(input_dim=
        X_train_tensor.shape[1])
    optimizer = optim.Adam(model.parameters(), lr=
        exp["lr"])

    # configurar el dataloader
    train_loader = DataLoader(train_dataset,
        batch_size=exp["batch"], shuffle=True)

    train_losses = []
    val_losses = []

    # ciclo de entrenamiento (epochs)
    for epoch in range(exp["epochs"]):
        model.train()
        epoch_train_loss = 0

        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()
            outputs = model(batch_X)

            # forzamos que el target sea [Batch, 1]
            batch_y_correct = batch_y.view(-1, 1).
                float()
            loss = criterion(outputs,
                batch_y_correct)

            loss.backward()
            optimizer.step()
            epoch_train_loss += loss.item()

        train_losses.append(epoch_train_loss / len(
            train_loader))

        # evaluacion en validacion para la curva
        model.eval()
        with torch.no_grad():
            val_outputs = model(X_val_tensor)
            y_val_correct = y_val_tensor.view(-1, 1)
                .float()
            val_loss = criterion(val_outputs,
                y_val_correct)
            val_losses.append(val_loss.item())

    # guardar el historial para graficas
    loss_history[f"Exp_{i+1}"] = {"train":
        train_losses, "val": val_losses}

    # evaluacion final para metricas comparativas
    model.eval()
    with torch.no_grad():
        logits = model(X_val_tensor)
        probs = torch.sigmoid(logits).numpy().
            flatten()
        preds = (probs >= 0.5).astype(int)

    y_true = y_val_tensor.numpy()

    results.append({
        "Exp": i+1,
        "LR": exp["lr"],
```

```
        "Batch": exp["batch"],
        "Epochs": exp["epochs"],
        "Accuracy": accuracy_score(y_true, preds),
        "Precision": precision_score(y_true, preds,
            zero_division=0),
        "Recall": recall_score(y_true, preds,
            zero_division=0),
        "F1": f1_score(y_true, preds, zero_division
            =0),
        "AUC": roc_auc_score(y_true, probs)
    })

results_df = pd.DataFrame(results)
```

Listing 21: Ejecución de los diez entrenamientos y almacenamiento de Loss

Tabla II: Resultados de los 10 experimentos ordenados por F1 sobre validación

Exp	LR	Batch	Epochs	Accuracy	Precision	Recall	F1	AUC
2	0.001	32	20	0.7402	0.7778	0.7130	0.7440	0.8234
10	0.010	32	50	0.7206	0.7629	0.6852	0.7220	0.8286
6	0.005	64	20	0.7157	0.7500	0.6944	0.7212	0.8202
7	0.010	16	20	0.7157	0.7551	0.6852	0.7184	0.8277
9	0.010	64	20	0.7108	0.7475	0.6852	0.7150	0.8264
8	0.010	32	20	0.7108	0.7475	0.6852	0.7150	0.8294
5	0.005	32	20	0.7059	0.7400	0.6852	0.7115	0.8273
4	0.005	16	20	0.7059	0.7400	0.6852	0.7115	0.8269
1	0.001	16	20	0.6961	0.7300	0.6759	0.7019	0.8252
3	0.001	64	20	0.6618	0.7191	0.5926	0.6497	0.7052

VIII-C. Curvas de Aprendizaje

Al graficar el historial guardado en el paso anterior, se contrastó el error de entrenamiento frente al error de validación para detectar problemas de generalización.

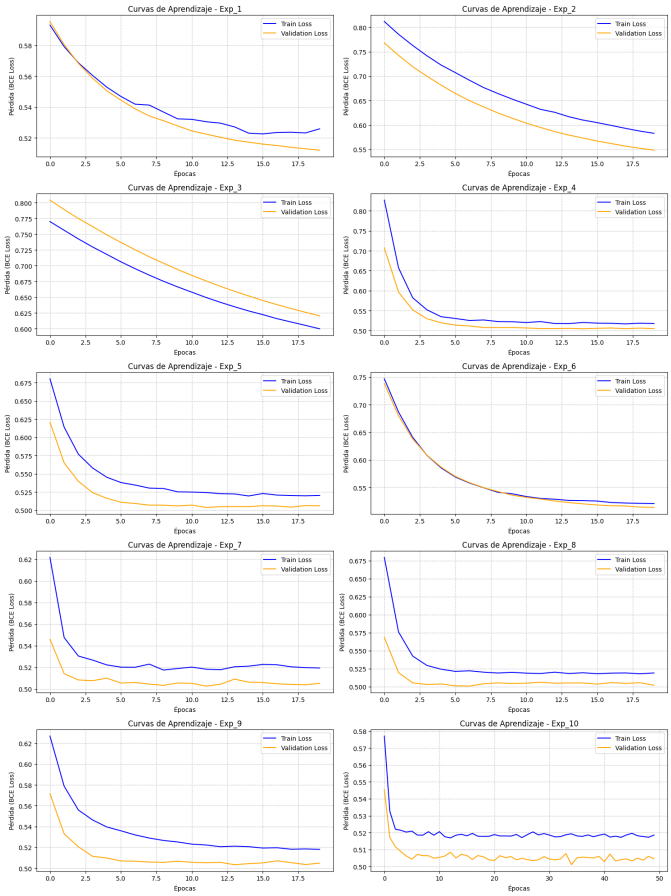

Figura 5: Gráficas de curvas de aprendizaje (Loss) para cada experimento.

El aspecto más destacable es el comportamiento de la curva de validación, la cual acompañó estrechamente el descenso de la curva de entrenamiento. El hecho de que el *Validation Loss* disminuyera y se estabilizara a la par del *Train Loss*, sin presentar repuntes abruptos o divergencias hacia el final de las épocas, es un indicador empírico sólido de que los modelos no sufrieron de sobreajuste (*overfitting*).

IX. Análisis de resultados y Selección del modelo

A partir del cuadro comparativo (Tabla II), se observó una varianza significativa en el rendimiento de los modelos. El **Experimento 2** destacó de manera contundente como el mejor modelo general. No solo obtuvo el máximo F1-Score (0.7440), sino que lideró casi todas las métricas de clasificación directa: Exactitud (0.7402), Precisión (0.7778) y Recall (0.7130). Su configuración ($LR = 0.001$, $Batch = 32$, $Epochs = 20$) logró el mejor punto de convergencia.

Se evidencia que un tamaño de lote de 32 fue el punto óptimo para este conjunto de datos, ya que los experimentos que lo utilizaron dominan la parte superior de la tabla. Por el contrario, tamaños de lote más grandes generaron actualizaciones de gradiente menos precisas, desplomando las métricas. Basado en esta evidencia cuantitativa, se seleccionó oficialmente el **Experimento 2** como el modelo definitivo.

IX-A. *Evaluación Final con Conjunto de Prueba (Testing Set)*

Para comprobar la capacidad real de generalización, se procedió a re-entrenar el modelo con la configuración ganadora y a evaluarlo utilizando el conjunto de prueba (Test Set).

```
# configuracion del modelo ganador
mejor_lr = 0.001
mejor_batch = 32
mejores_epochs = 20

modelo_final = LogisticRegressionModel(input_dim=
    X_train_tensor.shape[1])
optimizador_final = optim.Adam(modelo_final.
    parameters(), lr=mejor_lr)
criterio_final = nn.BCEWithLogitsLoss()

train_loader_final = DataLoader(train_dataset,
    batch_size=mejor_batch, shuffle=True)

# entrenamiento final
modelo_final.train()
for epoch in range(mejores_epochs):
    for batch_X, batch_y in train_loader_final:
        optimizador_final.zero_grad()
        outputs = modelo_final(batch_X)
        loss = criterio_final(outputs, batch_y.view
            (-1, 1).float())
        loss.backward()
        optimizador_final.step()

# evaluacion con el conjunto de testing
modelo_final.eval()
with torch.no_grad():
    test_logits = modelo_final(X_test_tensor)
    test_probs = torch.sigmoid(test_logits).numpy().
        flatten()
    test_preds = (test_probs >= 0.5).astype(int)
```

Listing 22: Re-entrenamiento y evaluación con el conjunto de pruebas

Al someter el modelo óptimo al conjunto de pruebas, se obtuvieron las siguientes métricas de rendimiento final: **Accuracy de 0.7549**, **Precision de 0.7900**, **Recall de 0.7315**, **F1-Score de 0.7596** y un **AUC de 0.8273**.

Al comparar estos resultados finales con los obtenidos durante la fase de validación (donde el F1-Score fue de 0.7440), se observó que no solo no hubo degradación del rendimiento, sino que el modelo logró una leve mejora en su capacidad de generalización.

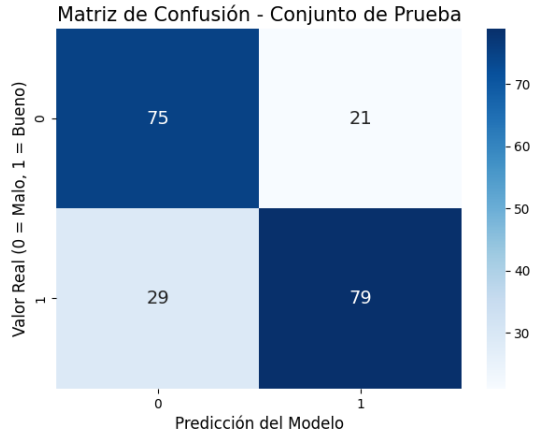

Figura 6: Matriz de confusión obtenida al evaluar el conjunto de pruebas.

La matriz de confusión (Figura 6) ofreció una visión detallada de los aciertos y errores sobre un total de 204 muestras en un escenario real. El modelo clasificó de forma correcta 79 vinos de buena calidad (TP) y 75 vinos de baja calidad (TN). Esta alta concentración en la diagonal principal evidenció que las características químicas seleccionadas poseen el peso matemático suficiente para separar ambas clases. La proximidad métrica entre los errores Tipo I (21) y Tipo II (29) demostró un clasificador equilibrado y estable, producto del correcto tratamiento previo y el muestreo estratificado.

X. Conclusiones

A partir del análisis exploratorio se determinó que el dataset presentaba una estructura adecuada para trabajar un problema de clasificación binaria, aunque fue necesario eliminar 240 filas duplicadas y tratar valores atípicos mediante el uso del rango intercuartílico (IQR). También se comprobó que no existían valores nulos, lo cual simplificó el preprocesamiento inicial.

La transformación de la variable `quality` a `quality_binary` permitió replantear el problema como una clasificación entre vinos buenos y malos. A partir del estudio de correlación se seleccionaron las seis variables con la relación más fuerte: *alcohol*, *volatile acidity*, *total sulfur dioxide*, *sulphates*, *density* y *citric acid*.

La implementación con PyTorch permitió construir un modelo de regresión logística y entrenarlo mediante mini-batches, capturando historiales de pérdida que demostraron un descenso saludable sin indicios de sobreajuste.

Finalmente, entre los diez experimentos ejecutados, el modelo definitivo (Experimento 2) no solo se destacó durante la validación, sino que al enfrentarse a datos desconocidos del conjunto de pruebas logró un F1 de 0.7596 y una Exactitud de 0.7549. Esto demuestra que la regresión logística desarrollada en PyTorch cumple satisfactoriamente con el objetivo propuesto de catalogar la calidad del vino basándose de forma exclusiva en su perfil químico.

NOTA IMPORTANTE SOBRE FIGURAS Y CIERRE DEL TRABAJO

Para compilar correctamente este documento, es necesario que las figuras utilizadas estén en la misma carpeta del archivo `.tex`. En este caso, los nombres empleados fueron los siguientes:

```
notebook_md_37_0.png
notebook_md_45_0.png
notebook_md_49_0.png
notebook_md_54_0.png
curvas_aprendizaje.png
matriz_confusion.png
```